TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

Part 1

What a Train Sim World vehicle actually is

In TSC a vehicle is a blueprint graph; in TSW it is a small family of Unreal assets that split *what the vehicle is* from *what it looks like*. This part reads that family the way the game does, follows one key press to a turning wheel, and maps every TSC idea you already have onto its TSW counterpart.

In this part

1. Model and view: why the split exists
2. From a key press to a turning wheel
3. The RVD, field by field
4. Inside the RVM
5. Inside the RVV
6. The cooked-asset rule
7. The TSC map, in detail

Component lists and field values in this part were read from `RVM_ART_Thomas`, `RVV_ART_Thomas` and `RVD_ART_Thomas` in the running editor, with the Class 47 and the 12-ton van for comparison (**verified**).

1 · Model and view: why the split exists

The RVM (model) is created when the route loads and is never destroyed, even for trains parked off-map in the “vault”. The RVV (view) is spawned when the player comes near and destroyed when they go far. The RVA (audio) comes and goes with the view. That one design decision explains most of what follows:

- Anything that must remember state (throttle position, doors, lights, fuel) lives on the RVM, tagged `SaveGame`.
- The RVV can be heavy (the Class 47's has 221 components) because only a handful exist at once.
- Keyboard input goes to the RVM, not the RVV, so the AI driver and the HUD work when the visuals are gone.
- A control (VHID) is always two components: a logic half on the RVM and a clickable half on the RVV, linked by name.

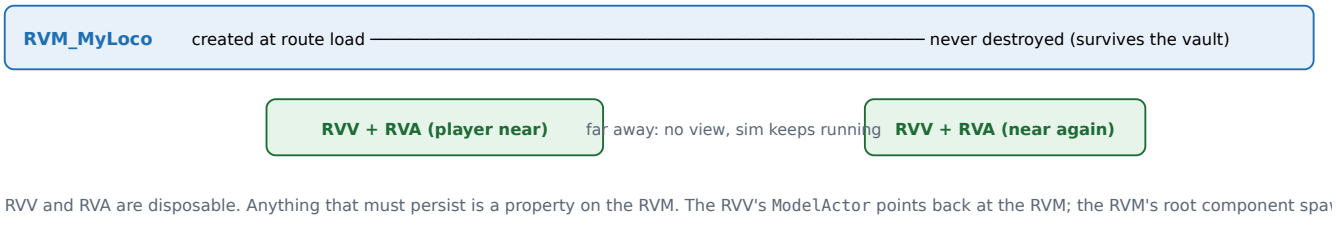


Figure 1.1 · Lifetimes. When you climbed into Thomas's cab, the driver pawn was attached to the RVV, which was attached to the RVM; both shared one world position. **verified**

2 · From a key press to a turning wheel

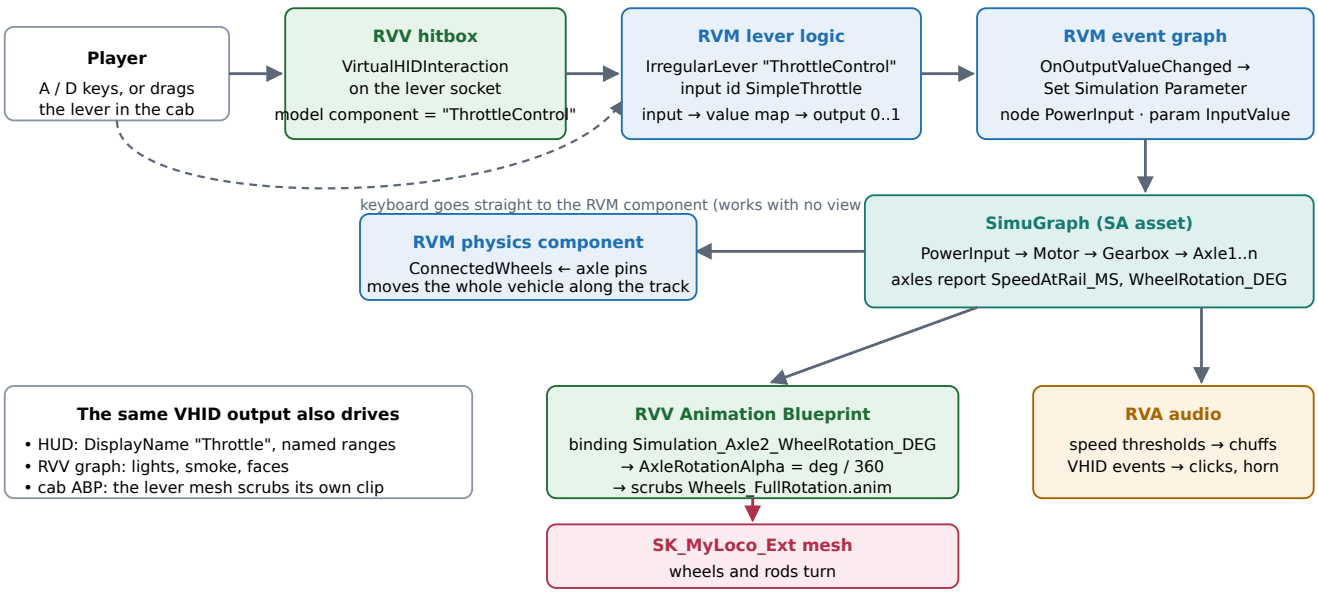


Figure 1.2 · The whole system on one page. Solid arrows are the main path; the dashed one is why keyboard input survives the view being unloaded. Every later part zooms into one box. **verified**

3 · The RVD, field by field

Created by right-click ▸ Miscellaneous ▸ Data Asset ▸ class `RailVehicleDefinition`. It is the only asset the rest of the game needs to know about; formations, timetables and the spawn menu reference nothing else. Thomas's values in the right column. **verified**

Property	What it does	Thomas
`ModelActorAsset` / `ViewActorAsset`	Required. Your RVM and RVV Blueprint classes.	RVM_ART_Thomas_C / RVV_…
`AdditionalViewActorAssets`	Extra view actors spawned with the RVV; the RVA goes here.	[RVA_ART_Thomas_C]
`VehicleCategory`	Locomotive, wagon, coach, multiple-unit car; drives filters and AI behaviour.	Locomotive
`bIsElectric / bIsDiesel / bIsSteam`	UI flags. Thomas has all three false although his description says “Steam”.	false ×3
`MaximumSpeed`	m/s. HUD and AI speed planning.	15.65 (35 mph)
`bUsesSimpleControls`	One combined throttle/brake lever scheme.	true
`ApproximateBounds / Offset / Length`	Centimetres, cached from the mesh; `bRegenBounds` recomputes. Used for spawning and vault packing.	(654, 242, 290)
`HUDMetersWidget`, `ThumbnailTexture`	HUD dial cluster; menu picture.	WBP_HUD_Artemis_C
`FriendlyName, ManufacturerName, EngineDescription, TypeDescription, CountryTableRow`	UI text; use the design-name form from the store guide.	“Thomas the Tank Engine”…
`Weight / TractiveEffort / PowerOutput UIValue`	Info-card numbers; cosmetic.	0
`Male/FemaleDriverOutfit, DriverCharacterCollection`	Crew you see in the cab (data assets).	DA_ART_Thomas_*
`ConnectivityData`	Per end: which couplers, hoses and jumper cables must match the neighbour. Auto-filled from the RVM's coupler components (`bRecalculateData`).	ThomasAndFriends_Hook, MR hose
`bIsDrivable, bIsSelectable, bIsAvailableForFormationEditor, bIsAvailableForUserScenarios`	Where the vehicle is offered.	true, true, true, false
`bIsSubstitutableUnit, CommonnessWeighting`	Let the game swap it in for a similar timetable vehicle; how often.	true, 0

Spawning needs a formation

Nothing spawns an RVD directly. A `TrainFormation` data asset (`FRM_`) lists RVDs in order; timetables, scenarios and the spawn menu pick formations. Make `FRM_MyLoco` containing just `RVD_MyLoco` so the loco can be placed on its own.

4 · Inside the RVM

The RVM is a Blueprint whose parent is `RailVehicle`. It has no meshes: it is a bag of logic components plus an event graph. Thomas's has 21 components, four inherited: **verified**

Component	Class	Role
ModelChildActorComponent0 (root)	inherited	Spawns the RVV (`view_actor_asset`) and the additional view actors.
RailVehiclePhysicsComponent0	inherited, class RailVehicleComponent	Size, mass, buffers, drag, couplers' parent, simple physics, VHID presets, events. You configure it, you do not add it.
GameplayTasks / GameplayTagStatus	inherited	Engine plumbing.
Simulation	SimulationComponent	Points at the SA_ SimuGraph. Name it exactly `Simulation` ; everything refers to it by that name.
Engineer'sEnvironment, Secondman'sEnvironment	InteractionEnvironmentComponent	Places a person can be (DRIVER / FRONT…). Controls, seats and cameras belong to one.
ThrottleControl, Reverser	IrregularLeverComponent	Levers: input → value map → output, notches.
Whistle, Headlights, Torch	PushButtonComponent	Push-and-hold or toggle buttons.
CabDoor_L / CabDoor_R	RailVehicleDoorComponent	Doors: state machine with lock, target state, transition time.
Coupler_F / Coupler_B, AirHose_F/B, JumperCable_F/B	Coupler / AirHose / JumperCable	Connection points; ids must match the neighbour's.
SeatManager	SeatManagerComponent	Who sits where; RVV seats register with it.

A wagon is a locomotive with the driving parts removed: the 12-ton van's RVM has the four inherited components, a Simulation, two couplers, two vacuum hoses, four door components, push buttons for handbrake and door locks, and a NumberingComponent. No throttle, no environments, no seats.

> **The RVM component list is the feature list**
>
> Adding a feature means adding a model component here, a matching hitbox on the RVV, and usually a few nodes in the event graph or the SimuGraph. That is the same shape as TSC's "control value + animation + Lua handler", just in three places you can see.

5 · Inside the RVV

Parent class `RailVehicleView`. Thomas's has 54 components (Class 08: 179; Class 47: 221), grouped by purpose: verified

Group	Components
Meshes	`SK_RailVehicle` (exterior skeletal mesh, ABP_..._Ext), `SK_Cab` (cab mesh, ABP_..._Cab), `CSX_Collision` (UCX static mesh), ground shadow, `Coupler_F/B` (ConnectionSkeletalMesh with a tiny ABP), `Blocker_F/B` boxes.
Interaction	One `VirtualHIDInteractionComponent` per control (CabDoor_L/R, Torch, Headlights...), `SeatComponent` s, `InteractionEnvironmentCameraComponent` s (driver, second man, head-out L/R), `CameraTransitionComponent` s (step up/down), auto-created tooltip anchors.
Lights and VFX	`LinearElectricLightComponent` controllers driving SpotLights, Niagara smoke, instanced wheel blur, auto-added RailVehicleParticleSystems (sparks, water spray, kick-up) at the `Wheel_*` sockets.
Housekeeping	`ComponentLODManagement` (turns components off by distance), audio asset manager.

This is where your Blender work lands: the two skeletal meshes, their animation blueprints, the collision mesh and the sockets that everything else attaches to. Parts 2 and 3 prepare them; Part 4 wires them in.

6 · The cooked-asset rule

Everything that ships with the game is *cooked*: editor data stripped. You can place, spawn and read Dovetail's vehicles at runtime, but you cannot open their Blueprint graphs or SimuGraphs, and Dovetail's FAQ confirms you cannot modify existing trains. Everything in this set about Dovetail's vehicles was read from the running game through Python (Part 6 §6), not by opening the assets. Your own plugin is uncooked source, so none of those limits apply to it. verified

7 · The TSC map, in detail

TSC	TSW	What changes for you
Engine blueprint: name, mass, length, max speed	RVD text fields; physics component `Node.Width/Height/Length/Mass`; RVD `MaximumSpeed`	Metres and kg on the RVM, centimetres in the RVD bounds. The build sheet (Part 7) prints them.
Bogey blueprints, pivots in XML	Bogie bones in the ABP's `Bogies` list; physics body tracks the rails	The engine yaws the bogie bones; the pivot is the bone position. XML had the final say in TSC; here the skeleton does.

TSC	TSW	What changes for you
Wheel nodes rotated by the game	`Axles` bone list or a one-revolution clip scrubbed by `WheelRotation_DEG`	Export `Wheels_FullRotation`; rods ride along in the same clip.
Control values (Regulator, Reverser…)	VHID output values	Same idea: a named 0..1 (or −1..1) number everything listens to.
.ia animation 0..N frames = control value 0..1	Clip scrubbed at time = VHID output	Direct translation; Part 3.
Sim blueprint: tractive effort curve, brakes	SimuGraph nodes; `SimplePhysics` block as fallback	Build a minimal graph first (Part 5), grow it later.
Lua: `Call("GetControlValue")`, `OnControlValueChange`, timers	Event graph: bind `OnOutputValueChanged`, Set/Get Simulation Parameter, `OnPreSimulate`	Visual nodes instead of text; physics logic moves into SimuGraph nodes.
Cab view .igs with its own camera	`SK_Cab` on the same skeleton + an InteractionEnvironmentCamera	The cab is a second mesh, not a second vehicle.
Lights by material / light nodes	LinearElectricLight controllers + lights or emissive parameters	Driven by a Headlights VHID through the RVV graph.
Sound blueprints	RVA actor with Sound Cues, curves, thresholds	Optional at first; two cues make a boilerplate.
Preload / scenario placing	Formation data asset	One per vehicle.